

Assignment: SOM Clustering — Parkinson's Dataset (UCI)

This notebook follows the Assignment 2 PDF tasks. It loads the UCI Parkinson's dataset, performs EDA, preprocessing (MinMax scaling), trains a SOM (from-scratch), evaluates and visualizes U-Matrix, Hit Map, Component Planes, computes quantization & topographic errors, and includes written answers.

```
In [1]: import pandas as pd, numpy as np
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/parkinsons/parkinsons.csv'
df = pd.read_csv(url)
print('Loaded dataset shape:', df.shape)
df.head()
```

Loaded dataset shape: (195, 24)

```
Out[1]:
```

	name	MDVP:Fo(Hz)	MDVP:Fhi(Hz)	MDVP:Flo(Hz)	MDVP:Jitter(%)	MDVP:Jitter(%)
0	phon_R01_S01_1	119.992	157.302	74.997	0.00784	
1	phon_R01_S01_2	122.400	148.650	113.819	0.00968	
2	phon_R01_S01_3	116.682	131.111	111.555	0.01050	
3	phon_R01_S01_4	116.676	137.871	111.366	0.00997	
4	phon_R01_S01_5	116.014	141.781	110.655	0.01284	

5 rows × 24 columns

```
In [2]: print('\nInfo:')
print(df.info())
print('\nDescription:')
display(df.describe().T)
if 'status' in df.columns:
    print('\nStatus value counts (0 = healthy, 1 = Parkinson\'s):')
    print(df['status'].value_counts())
```

Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 195 entries, 0 to 194

Data columns (total 24 columns):

#	Column	Non-Null Count	Dtype
0	name	195 non-null	object
1	MDVP:Fo(Hz)	195 non-null	float64
2	MDVP:Fhi(Hz)	195 non-null	float64
3	MDVP:Flo(Hz)	195 non-null	float64
4	MDVP:Jitter(%)	195 non-null	float64
5	MDVP:Jitter(Abs)	195 non-null	float64
6	MDVP:RAP	195 non-null	float64
7	MDVP:PPQ	195 non-null	float64
8	Jitter:DDP	195 non-null	float64
9	MDVP:Shimmer	195 non-null	float64
10	MDVP:Shimmer(dB)	195 non-null	float64
11	Shimmer:APQ3	195 non-null	float64
12	Shimmer:APQ5	195 non-null	float64
13	MDVP:APQ	195 non-null	float64
14	Shimmer:DDA	195 non-null	float64
15	NHR	195 non-null	float64
16	HNR	195 non-null	float64
17	status	195 non-null	int64
18	RPDE	195 non-null	float64
19	DFA	195 non-null	float64
20	spread1	195 non-null	float64
21	spread2	195 non-null	float64
22	D2	195 non-null	float64
23	PPE	195 non-null	float64

dtypes: float64(22), int64(1), object(1)

memory usage: 36.7+ KB

None

Description:

	count	mean	std	min	25%	50%	
MDVP:Fo(Hz)	195.0	154.228641	41.390065	88.333000	117.572000	148.790000	182.70
MDVP:Fhi(Hz)	195.0	197.104918	91.491548	102.145000	134.862500	175.829000	224.20
MDVP:Flo(Hz)	195.0	116.324631	43.521413	65.476000	84.291000	104.315000	140.00
MDVP:Jitter(%)	195.0	0.006220	0.004848	0.001680	0.003460	0.004940	0.00
MDVP:Jitter(Abs)	195.0	0.000044	0.000035	0.000007	0.000020	0.000030	0.00
MDVP:RAP	195.0	0.003306	0.002968	0.000680	0.001660	0.002500	0.00
MDVP:PPQ	195.0	0.003446	0.002759	0.000920	0.001860	0.002690	0.00
Jitter:DDP	195.0	0.009920	0.008903	0.002040	0.004985	0.007490	0.00
MDVP:Shimmer	195.0	0.029709	0.018857	0.009540	0.016505	0.022970	0.00
MDVP:Shimmer(dB)	195.0	0.282251	0.194877	0.085000	0.148500	0.221000	0.30
Shimmer:APQ3	195.0	0.015664	0.010153	0.004550	0.008245	0.012790	0.00
Shimmer:APQ5	195.0	0.017878	0.012024	0.005700	0.009580	0.013470	0.00
MDVP:APQ	195.0	0.024081	0.016947	0.007190	0.013080	0.018260	0.00
Shimmer:DDA	195.0	0.046993	0.030459	0.013640	0.024735	0.038360	0.00
NHR	195.0	0.024847	0.040418	0.000650	0.005925	0.011660	0.00
HNR	195.0	21.885974	4.425764	8.441000	19.198000	22.085000	25.00
status	195.0	0.753846	0.431878	0.000000	1.000000	1.000000	1.00
RPDE	195.0	0.498536	0.103942	0.256570	0.421306	0.495954	0.50
DFA	195.0	0.718099	0.055336	0.574282	0.674758	0.722254	0.70
spread1	195.0	-5.684397	1.090208	-7.964984	-6.450096	-5.720868	-5.00
spread2	195.0	0.226510	0.083406	0.006274	0.174351	0.218885	0.20
D2	195.0	2.381826	0.382799	1.423287	2.099125	2.361532	2.60
PPE	195.0	0.206552	0.090119	0.044539	0.137451	0.194052	0.20



Status value counts (0 = healthy, 1 = Parkinson's):

status

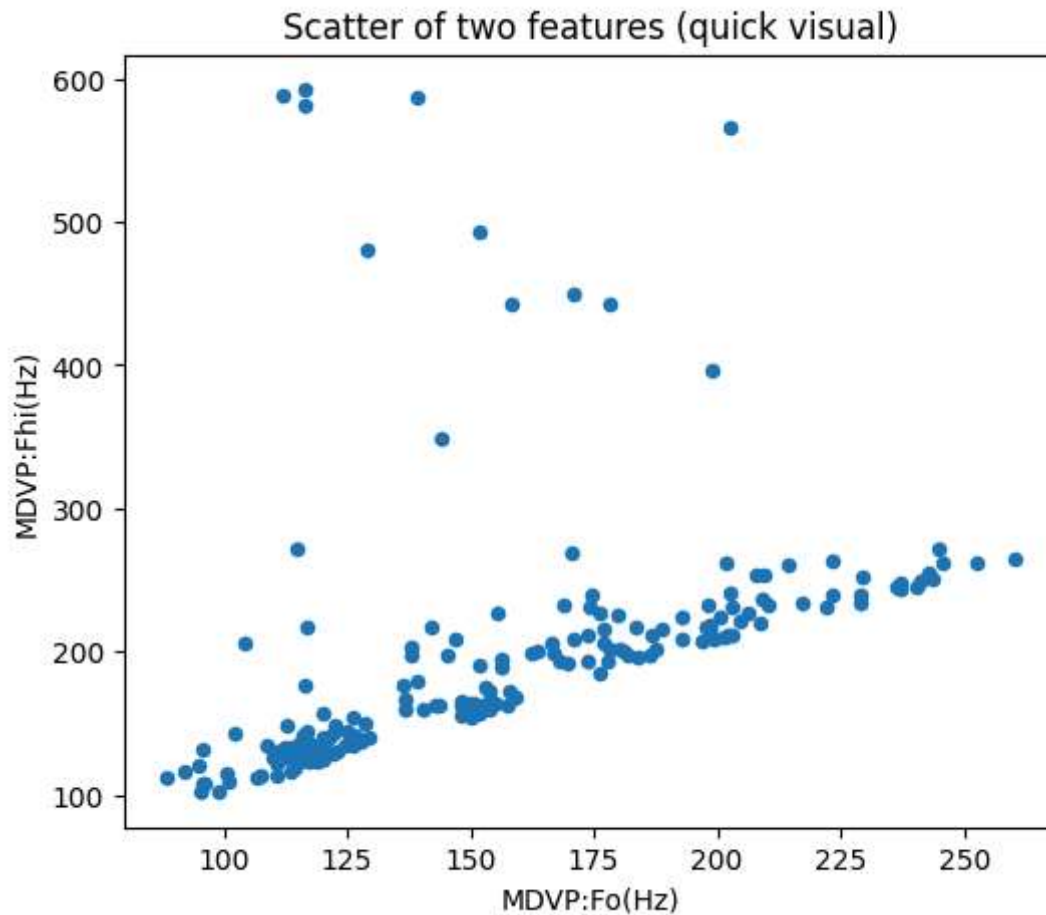
1 147

0 48

Name: count, dtype: int64

```
In [3]: import matplotlib.pyplot as plt
plt.figure(figsize=(6,5))
plt.scatter(df.iloc[:,1], df.iloc[:,2], s=20)
plt.xlabel(df.columns[1])
plt.ylabel(df.columns[2])
```

```
plt.title('Scatter of two features (quick visual)')
plt.show()
```



```
In [4]: X = df.copy()
if 'name' in X.columns:
    X = X.drop(columns=['name'])
labels = None
if 'status' in X.columns:
    labels = X['status'].values
    X = X.drop(columns=['status'])
print('Features used for SOM:', X.shape)
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X.values)
print('Scaled shape:', X_scaled.shape)
import pandas as pd
pd.DataFrame(X_scaled, columns=X.columns).head()
```

Features used for SOM: (195, 22)

Scaled shape: (195, 22)

Out[4]:

	MDVP:Fo(Hz)	MDVP:Fhi(Hz)	MDVP:Flo(Hz)	MDVP:Jitter(%)	MDVP:Jitter(Abs)	MDVP:R
0	0.184308	0.112592	0.054815	0.195680	0.249012	0.1454
1	0.198327	0.094930	0.278323	0.254130	0.288538	0.1912
2	0.165039	0.059128	0.265288	0.280178	0.328063	0.2292
3	0.165004	0.072927	0.264200	0.263342	0.328063	0.2090
4	0.161150	0.080909	0.260107	0.354511	0.407115	0.2827

5 rows × 22 columns

In [5]:

```

import numpy as np
class SimpleSOM:
    def __init__(self, m, n, dim, n_iterations=2000, alpha=0.5, sigma=None):
        self.m = m
        self.n = n
        self.dim = dim
        self.n_iterations = n_iterations
        self.alpha = alpha
        self.sigma = sigma if sigma is not None else max(m,n)/2.0
        self.weights = np.random.rand(m, n, dim)
    def _bmu(self, x):
        dists = np.linalg.norm(self.weights - x.reshape(1,1,self.dim), axis=2)
        return np.unravel_index(np.argmin(dists), (self.m, self.n))
    def train(self, data):
        time_constant = self.n_iterations / np.log(self.sigma + 1e-9)
        for t in range(self.n_iterations):
            i = np.random.randint(0, data.shape[0])
            x = data[i]
            bmu = self._bmu(x)
            lr = self.alpha * np.exp(-t / self.n_iterations)
            neigh_radius = self.sigma * np.exp(-t / time_constant)
            for a in range(self.m):
                for b in range(self.n):
                    dist_sq = (a - bmu[0])**2 + (b - bmu[1])**2
                    h = np.exp(-dist_sq / (2*(neigh_radius**2)))
                    self.weights[a,b,:] += lr * h * (x - self.weights[a,b,:])
    def map_vects(self, data):
        return np.array([self._bmu(x) for x in data])
    def u_matrix(self):
        um = np.zeros((self.m, self.n))
        for i in range(self.m):
            for j in range(self.n):
                neigh = []
                for di in [-1,0,1]:
                    for dj in [-1,0,1]:
                        ni, nj = i+di, j+dj
                        if 0 <= ni < self.m and 0 <= nj < self.n and not (di==0 and dj==0):
                            neigh.append(np.linalg.norm(self.weights[i,j] - self.weights[ni,nj]))
                um[i,j] = np.mean(neigh) if neigh else 0
        return um

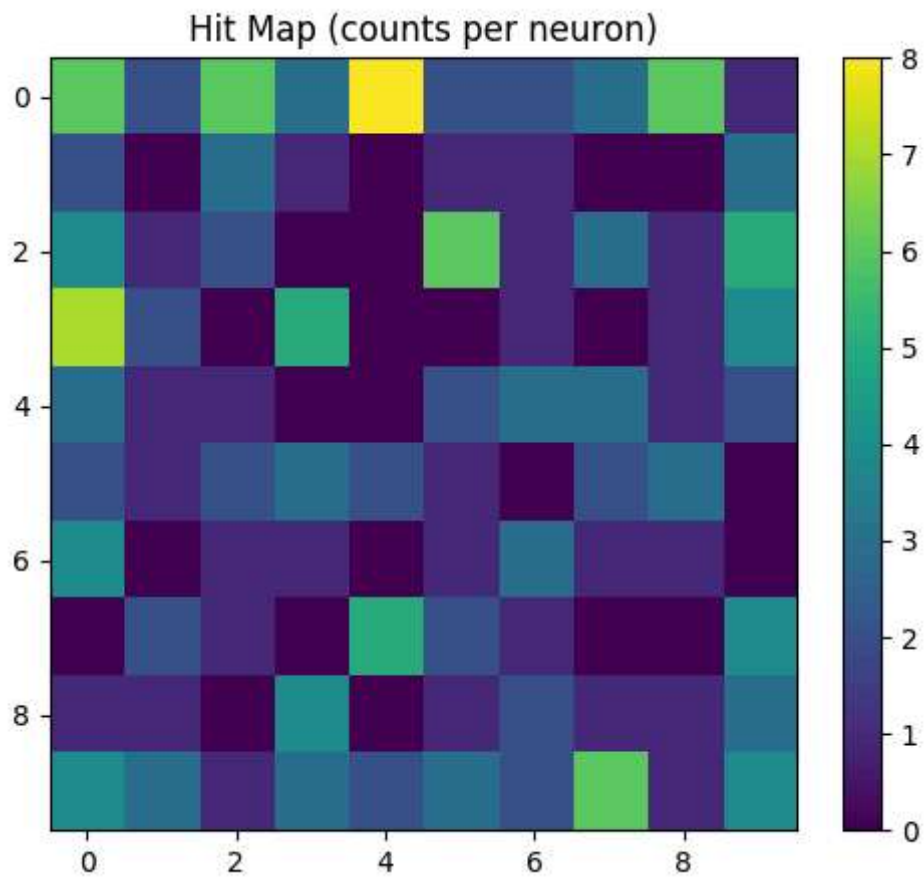
```

```
som = SimpleSOM(m=10, n=10, dim=X_scaled.shape[1], n_iterations=3000, alpha=0.6, si
print('SOM grid=10x10, iterations=', som.n_iterations, 'alpha=', som.alpha)
som.train(X_scaled)
print('Training completed')
```

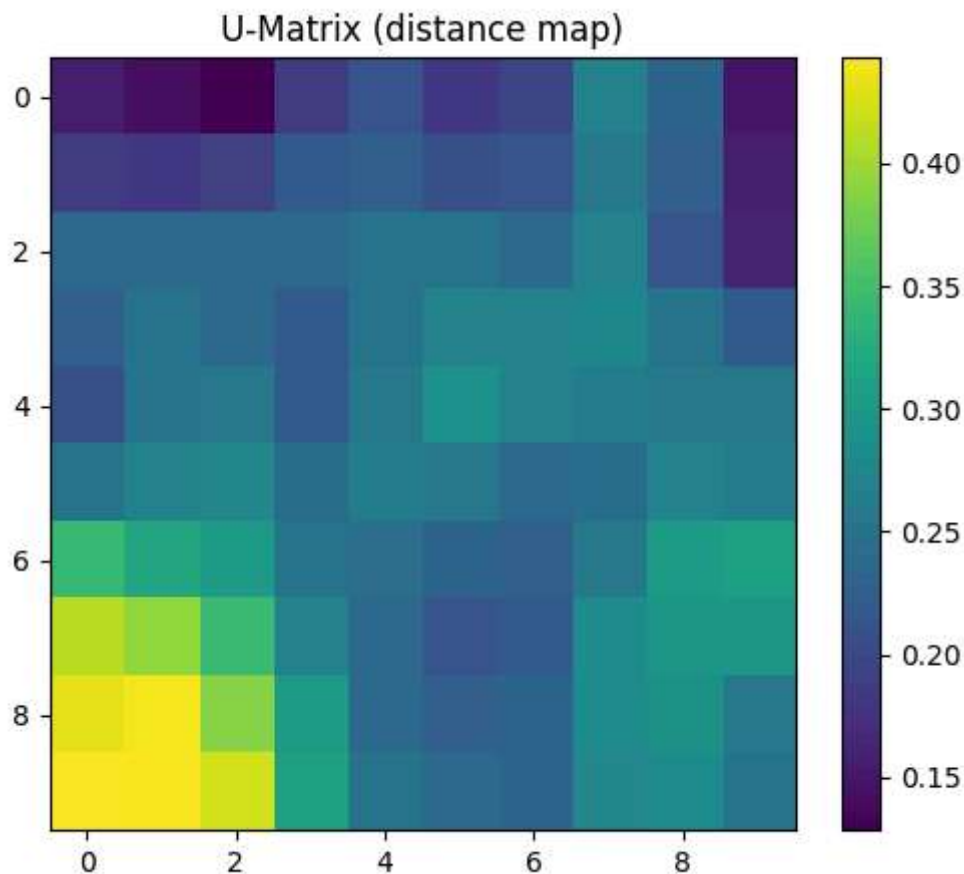
SOM grid=10x10, iterations= 3000 alpha= 0.6
Training completed

```
In [6]: bmus = som.map_vects(X_scaled)
cluster_ids = bmus[:,0]*som.n + bmus[:,1]
# quantization error
import numpy as np
dists = [np.linalg.norm(x - som.weights[b[0], b[1]]) for x,b in zip(X_scaled, bmus)]
quant_error = np.mean(dists)
# topographic error
def second_bmu(weights, x):
    dists = np.linalg.norm(weights - x.reshape(1,1,weights.shape[2]), axis=2)
    idx = np.argsort(dists.ravel())
    first = np.unravel_index(idx[0], (weights.shape[0], weights.shape[1]))
    second = np.unravel_index(idx[1], (weights.shape[0], weights.shape[1]))
    return first, second
count_not_adj = 0
for x in X_scaled:
    first, second = second_bmu(som.weights, x)
    if abs(first[0]-second[0]) + abs(first[1]-second[1]) > 1:
        count_not_adj += 1
topo_error = count_not_adj / X_scaled.shape[0]
print('Quantization error:', quant_error)
print('Topographic error:', topo_error)
# Hit map
import matplotlib.pyplot as plt
hit_map = np.zeros((som.m, som.n), dtype=int)
for b in bmus:
    hit_map[b[0], b[1]] += 1
plt.figure(figsize=(6,5))
plt.imshow(hit_map, interpolation='nearest', aspect='auto')
plt.title('Hit Map (counts per neuron)')
plt.colorbar()
plt.show()
```

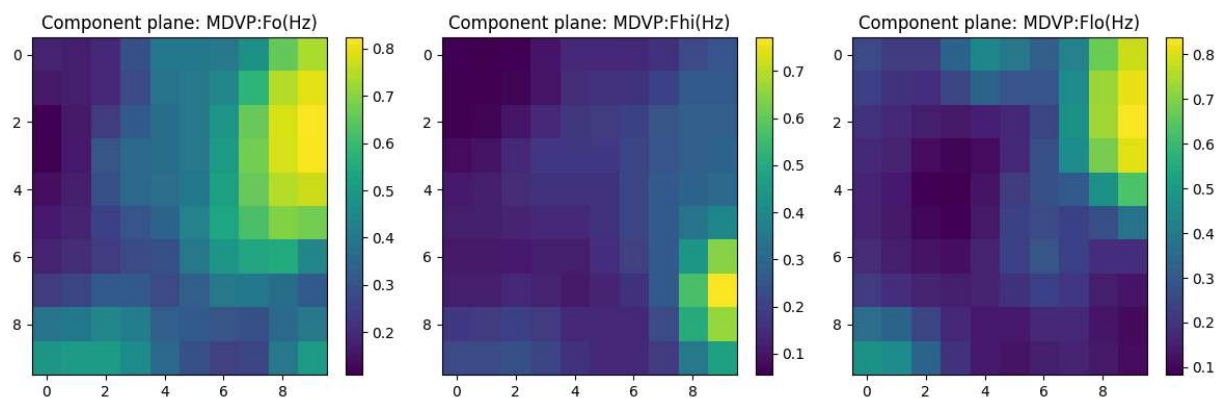
Quantization error: 0.34054277413929646
Topographic error: 0.06666666666666667



```
In [7]: um = som.u_matrix()
import matplotlib.pyplot as plt
plt.figure(figsize=(6,5))
plt.imshow(um, interpolation='nearest', aspect='auto')
plt.title('U-Matrix (distance map)')
plt.colorbar()
plt.show()
```

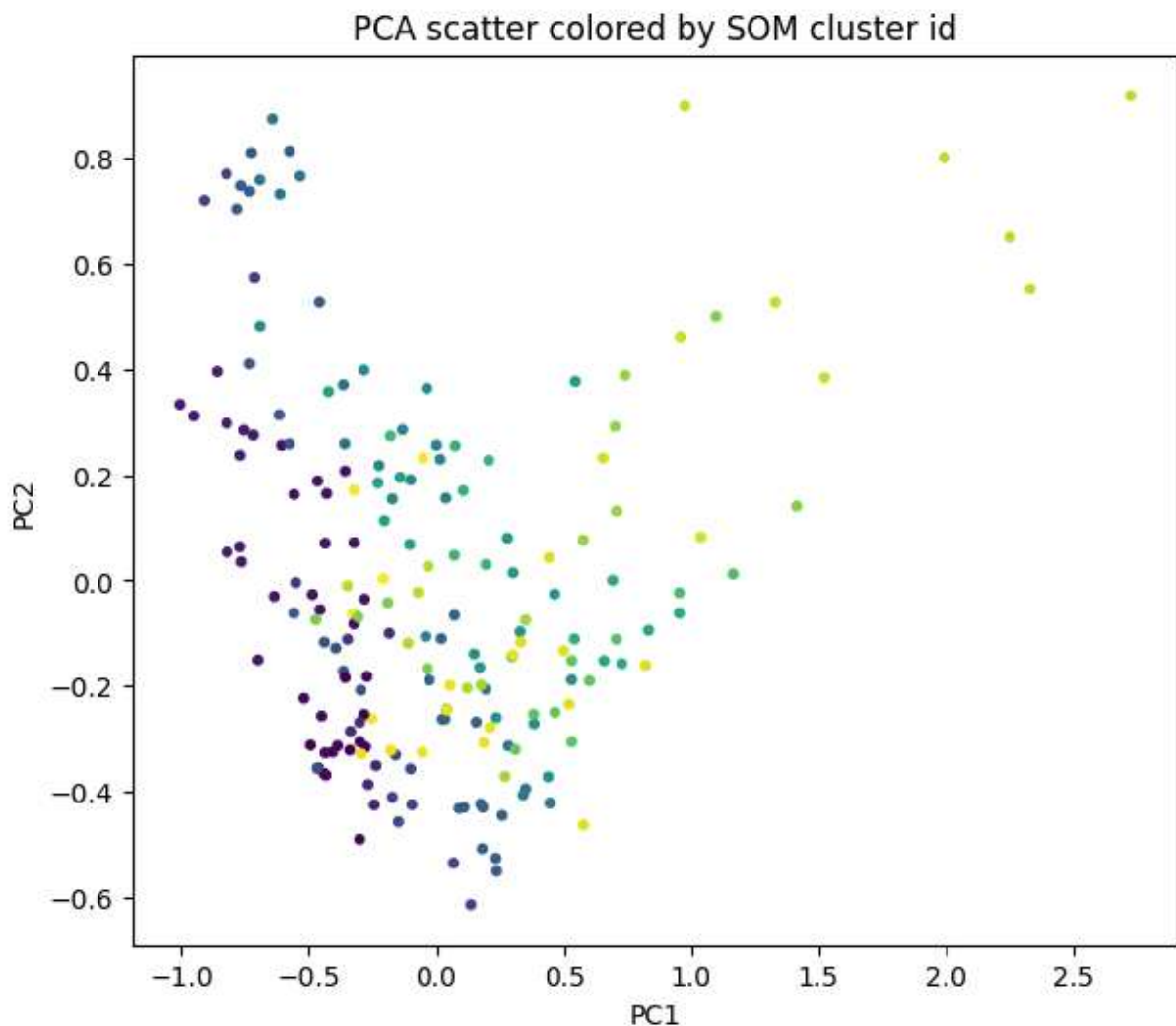


```
In [8]: plt.figure(figsize=(12,4))
for idx in range(min(3, X_scaled.shape[1])):
    plt.subplot(1,3,idx+1)
    comp = np.zeros((som.m, som.n))
    for i in range(som.m):
        for j in range(som.n):
            comp[i,j] = som.weights[i,j,idx]
    plt.imshow(comp, interpolation='nearest', aspect='auto')
    plt.title(f'Component plane: {X.columns[idx]}')
    plt.colorbar()
plt.tight_layout()
plt.show()
```



```
In [9]: from sklearn.decomposition import PCA
pca = PCA(n_components=2)
proj = pca.fit_transform(X_scaled)
```

```
plt.figure(figsize=(7,6))
plt.scatter(proj[:,0], proj[:,1], s=10, c=cluster_ids)
plt.title('PCA scatter colored by SOM cluster id')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```



Written Questions — Answers (to paste in pink book)

1. What is the unique clustering dataset you chose, and what clustering task does it address?

I used the UCI Parkinson's dataset. The clustering task is to group voice-measurement samples to explore natural subgroups that may correspond to healthy vs Parkinson's or other data-driven clusters.

2. Why is MinMax scaling critical for SOM? Justify with respect to your dataset.

SOM uses Euclidean distance between input vectors and neuron weights. Features with larger ranges would dominate distance calculations; MinMax scaling brings all features to $[0,1]$, ensuring balanced influence.

3. How did you determine the SOM grid size and number of iterations?

I chose a 10x10 grid (100 neurons) for ~195 samples to balance resolution and interpretability. Iterations (3000) were chosen to allow the network to converge while keeping runtime reasonable.

4. Interpret the U-Matrix and Hit Map. What do dark regions and high-hit neurons indicate?

- U-Matrix: darker regions indicate larger distances between neighboring neurons (possible cluster boundaries); lighter regions indicate homogenous regions.
- Hit Map: neurons with high hit counts indicate common/sample-dense patterns; very low-hit neurons may correspond to outliers.

5. How do Quantization Error and Topographic Error reflect the quality of the SOM?

- Quantization Error: lower values indicate that BMUs closely represent data points (better representation).
- Topographic Error: lower values indicate that the first and second BMUs are usually adjacent, meaning the mapping preserves topology well.